torch.load#

https://pytorch.org/docs/stable/generated/torch.load.html#torch.load

Description:

Loads an object saved with torch.save() from a file. torch.load() uses Python’s unpickling facilities but treats storages, which underlie tensors, specially. They are first deserialized on the CPU and are then moved to the device they were saved from. If this fails (e.g. because the run time system doesn’t have certain devices), an exception is raised. However, storages can be dynamically remapped to an alternative set of devices using the map_location argument. If map_location is a callable, it will be called once for each serialized storage with two arguments: storage and location. The storage argument will be the initial deserialization of the storage, residing on the CPU. Each serialized storage has a location tag associated with it which identifies the device it was saved from, and this tag is the second argument passed to map_location. The builtin location tags are 'cpu' for CPU tensors and 'cuda:device_id' (e.g. 'cuda:2') for CUDA tensors. map_location should return either None or a storage. If map_location returns a storage, it will be used as the final deserialized object, already moved to the right device. Otherwise, torch.load() will fall back to the default behavior,

Function Signature

```
torch.load(f, map_location=None, pickle_module=pickle, *,
weights_only=True, mmap=None, **pickle_load_args)[source]#

Parameters

Return type
```

Returns:

Any

Code Examples

```
>>> torch.load("tensors.pt", weights_only=True)
# Load all tensors onto the CPU
>>> torch.load(
...     "tensors.pt",
...     map_location=torch.device("cpu"),
...     weights_only=True,
... )
# Load all tensors onto the CPU, using a function
>>> torch.load(
...     "tensors.pt",
...     map_location=lambda storage, loc: storage,
...     weights_only=True,
... )
# Load all tensors onto GPU 1
```

```
>>> torch.load(
...     "tensors.pt",
...     map_location=lambda storage, loc: storage.cuda(1),
...     weights_only=True,
... )  # type: ignore[attr-defined]
# Map tensors from GPU 1 to GPU 0
>>> torch.load(
...     "tensors.pt",
...     map_location={"cuda:1": "cuda:0"},
...     weights_only=True,
... )
# Load tensor from io.BytesIO object
# Loading from a buffer setting weights_only=False, warning this
can be unsafe
>>> with open("tensor.pt", "rb") as f:
...     buffer = io.BytesIO(f.read())
>>> torch.load(buffer, weights_only=False)
# Load a module with 'ascii' encoding for unpickling
# Loading from a module setting weights_only=False, warning this
can be unsafe
>>> torch.load("module.pt", encoding="ascii",
weights_only=False)
```

Notes & Warnings

⚠ WARNING Warning torch.load() unless weights_only parameter is set to True, uses pickle module implicitly, which is known to be insecure. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling. Never load data that could have come from an untrusted source in an unsafe mode, or that could have been tampered with. Only load data you trust.

NOTE Note When you call torch.load() on a file which contains GPU tensors, those tensors will be loaded to GPU by default. You can call torch.load(.., map_location='cpu') and then load_state_dict() to avoid GPU RAM surge when loading a model checkpoint.

NOTE Note By default, we decode byte strings as utf-8. This is to avoid a common error case UnicodeDecodeError: 'ascii' codec can't decode byte 0x... when loading files saved by Python 2 in Python 3. If this default is incorrect, you may use an extra encoding keyword argument to specify how these objects should be loaded, e.g., encoding='latin1' decodes them to strings using latin1 encoding, and encoding='bytes' keeps them as byte arrays which can be decoded later with byte_array.decode(...).

Detailed Documentation

Documentation

Loads an object saved with torch.save() from a file.

torch.load() uses Python’s unpickling facilities but treats storages, which underlie tensors, specially. They are first deserialized on the CPU and are then moved to the device they were saved from. If this fails (e.g. because the run time system doesn’t have certain devices), an exception is raised. However, storages can be dynamically remapped to an alternative set of devices using the map_location argument.

If map_location is a callable, it will be called once for each serialized storage with two arguments: storage and location. The storage argument will be the initial deserialization of the storage, residing on the CPU. Each serialized storage has a location tag associated with it which identifies the device it was saved from, and this tag is the second argument passed to map_location. The builtin location tags are 'cpu' for CPU tensors and 'cuda:device_id' (e.g. 'cuda:2') for CUDA tensors. map_location should return either None or a storage. If map_location returns a storage, it will be used as the final deserialized object, already moved to the right device. Otherwise, torch.load() will fall back to the default behavior, as if map_location wasn’t specified.

If map_location is a torch.device object or a string containing a device tag, it indicates the location where all tensors should be loaded.

Otherwise, if map_location is a dict, it will be used to remap location tags appearing in the file (keys), to ones that specify where to put the storages (values).

User extensions can register their own location tags and tagging and deserialization methods using torch.serialization.register_package().

See Layout Control for more advanced tools to manipulate a checkpoint.

f (Union[str, PathLike[str], IO[bytes]]) – a file-like object (has to implement read(),

readline(), tell(), and seek()), or a string or os.PathLike object containing a file name

map_location (Optional[Union[Callable[[Storage, str], Storage], device, str, dict[str, str]]]) – a function, torch.device, string or a dict specifying how to remap storage locations

pickle_module (Optional[Any]) – module used for unpickling metadata and objects (has to match the pickle_module used to serialize file)

weights_only (Optional[bool]) – Indicates whether unpickler should be restricted to loading only tensors, primitive types, dictionaries and any types added via torch.serialization.add_safe_globals(). See torch.load with weights_only=True for more details.

mmap (Optional[bool]) – Indicates whether the file should be mapped rather than loading all the storages into memory. Typically, tensor storages in the file will first be moved from disk to CPU memory, after which they are moved to the location that they were tagged with when saving, or specified by map_location. This second step is a no-op if the final location is CPU. When the mmap flag is set, instead of copying the tensor storages from disk to CPU memory in the first step, f is mapped, which means tensor storages will be lazily loaded when their data is accessed.

pickle_load_args (Any) – (Python 3 only) optional keyword arguments passed over to pickle_module.load() and pickle_module.Unpickler(), e.g., errors=....

torch.load() unless weights_only parameter is set to True, uses pickle module implicitly, which is known to be insecure. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling. Never load data that could have come from an untrusted source in an unsafe mode, or that could have been tampered with. Only load data you trust.

When you call torch.load() on a file which contains GPU tensors, those tensors will be loaded to GPU by default. You can call torch.load(.., map_location='cpu') and then

load_state_dict() to avoid GPU RAM surge when loading a model checkpoint.

By default, we decode byte strings as utf-8. This is to avoid a common error case UnicodeDecodeError: 'ascii' codec can't decode byte 0x... when loading files saved by Python 2 in Python 3. If this default is incorrect, you may use an extra encoding keyword argument to specify how these objects should be loaded, e.g., encoding='latin1' decodes them to strings using latin1 encoding, and encoding='bytes' keeps them as byte arrays which can be decoded later with byte_array.decode(...).
